

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Programação Concorrente
Exame de Época Normal, Verão de 2024/2025
Duração: 2 horas

1. [2] Considere a classe **UnsafeStack** com uma implementação não *thread-safe* de um *stack*. Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.

```
class UnsafeStack<T> {  
    class Node<T>(val value: T, val next: Node<T>?)  
    private var head: Node<T>? = null  
    fun push(value: T) {  
        head = Node(value, head)  
    }  
    fun pop(): T? {  
        val observedHead = head  
        return if(observedHead != null) {  
            head = observedHead.next  
            observedHead.value  
        } else {  
            null  
        }  
    }  
}
```

2. [5] Realize o sincronizador **CyclicCountDownLatch** com a interface apresentada em seguida.

```
class CyclicCountDownLatch(val initialCount: Int) {  
    init { require(initialCount > 0) }  
    @Throws(InterruptedException::class)  
    fun countDownAndAwait(timeout: Duration): Boolean  
}
```

A função **countDownAndAwait** decrementa o contador e espera que este chegue a zero. Se o contador chegar a zero, deve ser reposto de imediato com **initialCount**, e as *threads* bloqueadas em **countDownAndAwait** devem terminar a espera, retornando todas o valor **true**. A função **countDownAndAwait** retorna **false** caso o tempo máximo de espera seja atingido, sendo também sensível à interrupção da *thread* em que for chamada, tratando-a de acordo com o protocolo definido na plataforma Java.

3. [4] Realize o sincronizador **CyclicCountDownLatch**, para utilizar em corrotinas, com a interface apresentada em seguida.

```
class CyclicCountDownLatch(val initialCount: Int) {  
    init { require(initialCount > 0) }  
    suspend fun countDownAndAwait(): Unit  
}
```

A função **countDownAndAwait** decrementa o contador e espera que este chegue a zero. Se o contador chegar a zero, deve ser reposto de imediato com **initialCount** e todas as corrotinas que estejam nesse momento em espera em **countDownAndAwait** devem terminar a espera. A função **countDownAndAwait** não suporta desistência nem cancelamento.

4. [5] Considere a seguinte definição, que representa uma mensagem que deve ser apresentada ciclicamente com intervalos de **delayBetweenPresentations**.

```
class PeriodicMessage(val message: String, val delayBetweenPresentations: Duration)
```

Realize a seguinte função *non-suspend*

```
fun show(showString: (String) -> Unit, totalDuration: Duration,  
        vararg periodicMessages: PeriodicMessage)
```

A função **show** apresenta as mensagens **periodicMessages**, durante o tempo máximo de **totalDuration**, após o qual deve retornar. A função **showString** deve ser usada para apresentar cada uma das *strings*. Assuma que a função **showString** não é *thread-safe*, não podendo ser chamada simultaneamente a partir de múltiplas *threads*. A função **show** realizada deve ser *thread-safe*.

Ao contrário do segundo trabalho prático, deve usar funcionalidades do *package* **kotlinx.coroutines** para resolver esta questão.

5. [4] Considere a seguinte interface presente na biblioteca *standard* do Java

```
public interface Executor { void execute(Runnable command); }
```

Realize a função extensão

```
suspend fun <T1,T2> Executor.invoke(f1: ()->T1, f2: ()->T2): Pair<T1,T2>
```

Esta função promove a execução das funções **f1** e **f2** no *thread pool* representado pelo **Executor** passado como receptor, tirando partido das múltiplas *worker threads* que este pode ter. O resultado é um par com os resultados produzidos por cada uma das avaliações de **f1** e **f2**. A função **invoke** não deve bloquear a *thread* onde é chamada. Assuma que as avaliações de **f1** e **f2** não produzem exceções.

Duração: 2 horas
ISEL, 17 de junho de 2025